OFFICIAL ARCHITECTURE & VIVA DOCUMENTATION

# SkillHub - Enterprise Learning & Course Management Platform

## Master Project Documentation, Architecture Guide & Viva Cheatsheet

## 1. Executive Summary & Project Vision

**SkillHub** is an enterprise-grade, full-stack digital learning management platform designed and built using **ASP.NET Core 8 Web API** and **Angular 20 (Standalone Architecture)**. The platform provides an end-to-end ecosystem catering to five distinct personas: **Students**, **Instructors**, **SubAdmins**, **SuperAdmins**, and **Product Support Specialists**.

### Core Platform Capabilities

- **Curriculum & Course Authoring**: Instructors build multi-section courses with streaming video lessons, downloadable PDF resources, and structured syllabi.
- **Interactive Learning Player**: Students stream video lectures, toggle completion status, track progress in real time, and download reference materials.
- **Mastery Assessments & 70% Rule Certification**: Automated quiz engine with multiple-choice questions. Certificates are locked until the student finishes **100% of lessons** and achieves **≥ 70% score** on all assessments.
- **Enterprise Helpdesk & Support Triage**: Role-governed ticketing system where **Product Support** exclusively triages tickets, assigns them to Admins, or solves them directly, while **SuperAdmins** have strict read-only oversight.
- **Permission Studio (RBAC + PBAC)**: Dynamic permission management where platform privileges can be inspected and updated without recompiling or redeploying code.
- **Google Gemini AI Learning Copilot**: Integrated AI tutor providing contextual learning guidance, smart prompt suggestions, and student assistance with guest quota controls.
- **Integrated Payment Checkout**: Bank-grade checkout gateway supporting Cards, UPI QR Code, and Net Banking with instant atomic enrollment unlocking.

## 2. System Architecture & Clean Architecture Deep-Dive

SkillHub adheres strictly to **Clean Architecture** (Onion / Hexagonal Architecture principles). The core tenet is the **Dependency Inversion Principle**: high-level business logic does not depend on low-level implementation details (such as database engines or UI frameworks); rather, both depend on abstractions.




### Why Clean Architecture Over Traditional 3-Tier Architecture?

1. **Separation of Concerns**: Business rules in `SkillHub.Application` and `SkillHub.Domain` are independent of SQL Server, Entity Framework, or ASP.NET Core controllers.
2. **Testability**: Core logic can be unit-tested without mocking a live database or spinning up web servers.
3. **Maintainability & Evolution**: If SQL Server is replaced with PostgreSQL, or if Angular is replaced with a mobile app (Flutter/MAUI), the Domain and Application layers remain untouched.

### Layer Breakdown in the Codebase

1. `SkillHub.Domain` **(Core Business Objects)**

- **Zero dependencies** on external libraries or frameworks.
- Contains enterprise business entities, value objects, and domain enums.

- **Key Files**:
  - `BaseEntity.cs`: Common audit fields (`Id`, `CreatedAt`, `UpdatedAt`, `CreatedBy`, `UpdatedBy`).
  - `Entities/Identity/`: `User.cs`, `Role.cs`, `UserRole.cs`, `Permission.cs`, `RolePermission.cs`, `UserPermission.cs`, `RefreshToken.cs`.
  - `Entities/Courses/`: `Course.cs`, `CourseSection.cs`, `CourseLesson.cs`, `LessonResource.cs`, `CourseCategory.cs`, `Enrollment.cs`.
  - `Entities/Learning/`: `LessonCompletion.cs`, `CourseReview.cs`, `Certificate.cs`.
  - `Entities/Assessments/`: `Quiz.cs`, `QuizQuestion.cs`, `QuizOption.cs`, `QuizAttempt.cs`, `QuizAnswer.cs`.
  - `Entities/Support/`: `SupportTicket.cs`, `TicketMessage.cs`.

2. `SkillHub.Application` **(Business Orchestration Contracts)**

- Defines the use cases, service contracts (interfaces), transfer objects (DTOs), mappings, and validation rules.
- **Key Files**:
  - `Features/Courses/DTOs/`: `CourseDtos.cs` (DTOs for creating, updating, and viewing courses).
  - `Features/Learning/DTOs/`: `LearningDtos.cs` (DTOs for player progress, lesson completion, certificates).
  - `Features/Assessments/DTOs/`: `QuizDtos.cs` (DTOs for quiz questions, submission, scoring).
  - `Features/Support/DTOs/`: `SupportDtos.cs` (DTOs for ticketing, status updates, assignment).
  - `Features/Ai/DTOs/`: `AiChatDtos.cs` (DTOs for Gemini requests, responses, suggestions).
  - `Mappings/MappingProfile.cs`: AutoMapper configuration mapping Domain Entities to DTOs.
  - `Features/*/Validators/`: FluentValidation classes (`CreateCourseValidator.cs`, `LoginRequestValidator.cs`, etc.).

3. `SkillHub.Infrastructure` **(Data Access, External Services & Persistence)**

- Implements the contracts defined in the Application layer.
- Handles EF Core data access, database migrations, database seeding, password hashing, and third-party integrations (Google Gemini, SMTP).
- **Key Files**:
  - `Persistence/Context/ApplicationDbContext.cs`: EF Core database context configuring all 19 DbSets and relationship constraints.
  - `Services/Learning/LearningService.cs`: Orchestrates lesson toggles, completion percentages, and certificate evaluation.
  - `Services/Assessments/QuizService.cs`: Manages quiz questions, calculates attempt scores, and records student attempts.
  - `Services/Support/SupportService.cs`: Handles ticket creation, staff messaging, status workflows, and admin assignment.
  - `Services/Ai/GeminiAiService.cs`: Google Gemini REST API client with streaming prompt engineering and guest quota caching.
  - `Services/Admin/PermissionService.cs`: Provides catalog data for Permission Studio and handles role-permission bindings.
  - `seed/`: `RoleSeeder.cs`, `UserSeeder.cs`, `PermissionSeeder.cs`, `CategorySeeder.cs`.

4. `SkillHub.API` **(Presentation & HTTP Delivery)**

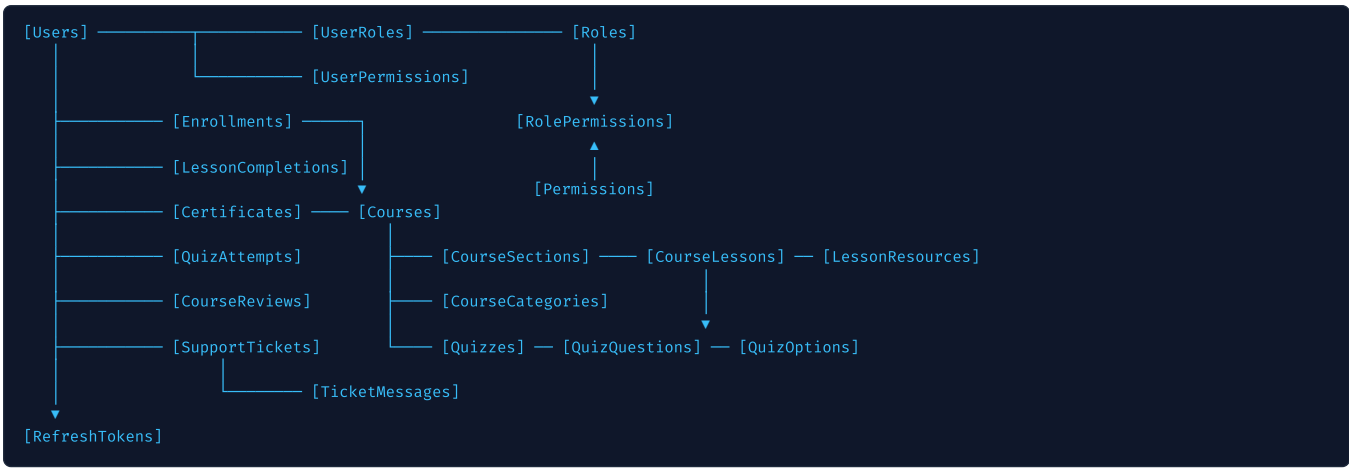
- Exposes RESTful endpoints, handles HTTP authentication headers, cross-origin resource sharing (CORS), and global exception handling.
- **Key Files**:
  - `Controllers/AuthController.cs`: Registration, login, refresh tokens, password reset.
  - `Controllers/CourseController.cs`: Course browsing, filtering, search, and authoring.
  - `Controllers/LearningController.cs`: Enrolled player data, lesson completion toggles.
  - `Controllers/QuizzesController.cs`: Quiz retrieval, answer submission, assessment scores.
  - `Controllers/SupportController.cs`: Ticket creation, queue viewing, status transitions, admin assignment.
  - `Controllers/AdminController.cs`: User governance, role assignment, course approvals.
  - `Controllers/PermissionController.cs`: Permission Studio endpoints.
  - `Controllers/AiChatController.cs`: Gemini AI interaction endpoint.
  - `Program.cs`: ASP.NET Core startup, dependency injection container setup, JWT configuration, CORS policy.

---

## 3. Database Architecture & What Makes It Unique

The database uses **Entity Framework Core 8 Code-First** with Microsoft SQL Server. It is engineered with strict relational integrity, defensive delete constraints, and audit capabilities.

### Entity Relationship Diagram (ERD Summary)




### What Makes Our Database Architecture Distinct?

| Unique Architecture Feature | How SkillHub Implements It | Real-World Benefit |
|---|---|---|
| **Defensive Delete Constraints** ( `OnDelete(DeleteBehavior.Restrict)` ) | Configured in `ApplicationDbContext.cs` for Users, Tickets, and Assigned Admins. | Prevents cascading deletes from accidentally wiping student learning history or ticket audit logs if a user is deleted. |
| **Hybrid RBAC + PBAC** | Normalized `RolePermissions` and `UserPermissions` join tables. | Enables both role grouping (e.g. `ProductSupport` ) and individual user permission overrides (e.g. granting a specific SubAdmin approval power). |
| **Strict Assessment Auditing** | `QuizAttempts` and `QuizAnswers` tables record exact historical attempts and timestamps. | Allows students to review their mistakes while preventing cheating or score manipulation. |
| **Deterministic Unique Ticket Numbers** | Unique indexed ticket numbers ( `SKILLHUB-TKT-XXXX` ) generated with random cryptographic suffixes. | Clean, human-readable identifiers for students and support agents. |
| **Automated Seeding Pipeline** | System seeds default roles, admin credentials, standard categories, and 30+ granular permissions on startup. | Zero-setup onboarding: the platform can spin up on a fresh database with fully working demo data. |

## 4. Role Governance & Access Control Matrix

SkillHub implements a role model tailored to enterprise operations:

| SKILLHUB ROLE GOVERNANCE | |
|---|---|
| Role | Primary Responsibilities & Scope |
| SuperAdmin | Platform owner. Full access to user management, course approvals, and Permission Studio.<br>*Special rule*: Read-only oversight on support tickets. |
| SubAdmin | Delegated operational administrator. Handles course approvals and works on support tickets assigned to them. |
| ProductSupport | Helpdesk dispatcher & query solver. Exclusively triages incoming tickets, resolves queries, and assigns tickets to administrators. Cannot access general admin tools. |
| Instructor | Content creator. Authors courses, defines syllabus sections, uploads video lessons & PDFs, and creates quizzes. |
| Student | Learner. Enrolls in courses, watches lectures, takes quizzes, downloads certificates, and opens support tickets. |

### Granular Support Desk Permissions Matrix

| Capability | Product Support | SubAdmin (Assigned) | SuperAdmin | Student |
|---|---|---|---|---|
| **View All Tickets** | ✅ Yes | ✅ Queue / Assigned | ✅ Yes (Read-Only) | ❌ Only Own |
| **Assign Ticket to Admin** | ✅ **Exclusive** | ❌ Blocked (403) | ❌ **Blocked (403)** | ❌ Blocked |
| **Change Ticket Status** | ✅ Yes | ✅ Yes | ❌ **Blocked (403)** | ❌ Blocked |
| **Post Staff Reply** | ✅ Yes | ✅ Yes | ❌ **Blocked (403)** | ❌ Only Student Msg |
| **Create Support Ticket** | ✅ Yes | ✅ Yes | ✅ Yes | ✅ Yes |

- **Backend Enforcement**: SupportController.cs
- **Frontend UI & Guards**: admin-support.ts, admin-support.html

---

## 5. Course System, Video Player & PDF Resources

### Course Lifecycle

```
[Instructor Creates Course] ──► [Draft State: IsPublished = false]
                                          │
                                          ▼ (Submits for Review)
                                [Admin / SubAdmin Reviews Course]
                                          │
                                          ▼ (Calls /api/Admin/courses/{id}/approve)
                                [Course Published: Visible in Marketplace]
```

### Routing to the Learning Player

- When a student completes checkout or clicks "Continue Learning", Angular directs them to: `http://localhost:4201/learn/:courseId`
- **Route Guard & Backend Verification**:
  1. Frontend calls `GET /api/Learning/courses/{courseId}`.
  2. `LearningService.cs` verifies whether the student has an active record in `Enrollments`.
  3. **Privilege Override**: If the user is a `SuperAdmin`, `SubAdmin`, or the course author (`InstructorId == userId`), the backend unlocks **Preview Mode** so staff can inspect content without purchasing.
  4. Non-enrolled students are rejected with `401 Unauthorized` and redirected to the course landing page.

### Player Capabilities

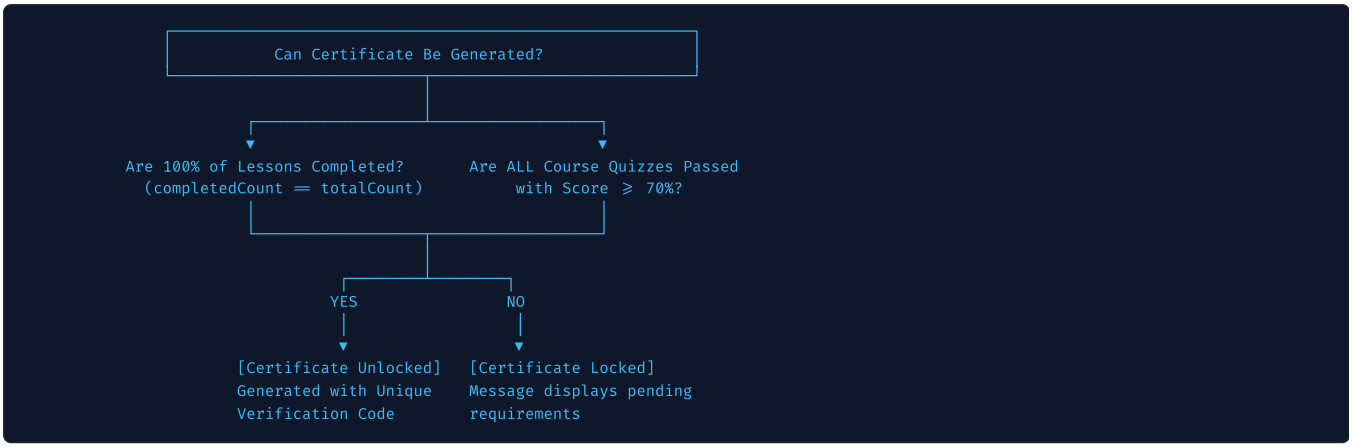
- **Video Playback**: Streams video URLs with automatic iframe or HTML5 video fallback.
- **Lesson Completion**: Toggle button updates `LessonCompletions` in real time, recalculates course completion percentage, and updates the circular progress bar.
- **Resource Downloads (PDFs)**:
  - Lessons can have multiple attachments stored in `LessonResources`.
  - Stored with `FileName` and `FileUrl`.
  - Students can click resource badges to open reference PDFs and worksheets directly in a new tab or trigger downloads.
- **Key Files**:
  - Backend: LearningController.cs, LearningService.cs
  - Frontend: course-player.ts, course-player.html, course-player.scss

---

## 6. Assessments, Quizzes & Strict 70% Certification Engine

### The Strict 70% Rule

To ensure academic integrity, certificates cannot be unlocked simply by clicking through video lessons. SkillHub enforces a two-tier validation rule in `LearningService.cs`:




### Grading Scale:

- **90% - 100%**: *Distinction*
- **80% - 89.9%**: *First Class with Distinction*
- **70% - 79.9%**: *Pass with Merit*
- **< 70%**: *Fail* (Must retake quiz to qualify for certificate)

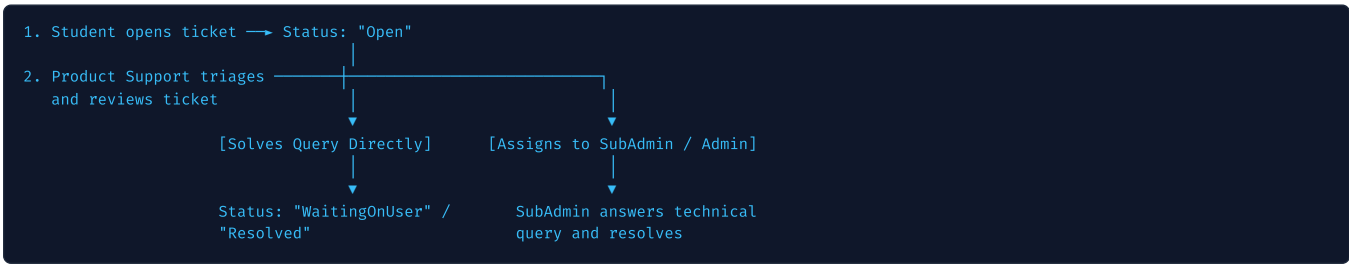
### Certificate Attributes:

- Unique certificate code: `SKILLHUB-CERT-XXXXXXXX` (UUID-backed alphanumeric identifier).
- Dynamic printable view with student name, course title, completion date, verified instructor name, grade, and security seals.
- **Key Files**:
  - Backend: LearningService.cs:L276-L380, QuizService.cs, Certificate.cs
  - Frontend: course-player.ts:L350-L420, quiz.model.ts

---

## 7. Helpdesk & Support Ticketing Workflow

### Support Ticket Architecture & Life Cycle




### Why SuperAdmin Has Read-Only Oversight:

In enterprise service desk operations, high-level executives and SuperAdmins maintain platform visibility to monitor SLA compliance, response times, and customer satisfaction metrics without interfering with day-to-day tier-1 and tier-2 operational queues.

- **Backend Enforcement**: In `SupportController.cs`:
  - `UpdateTicketStatus`: If `User.IsInRole("SuperAdmin")` returns `403 Forbidden`.
  - `AssignTicket`: If `!User.IsInRole("ProductSupport")` returns `403 Forbidden`.
  - `AddMessage`: If `User.IsInRole("SuperAdmin")` and caller is not ticket owner returns `403 Forbidden`.
- **Assignable Admins Endpoint**: `GET /api/Support/admin/assignable-admins` fetches all active administrators (`SubAdmin` and `SuperAdmin`), enabling Product Support agents to route tickets dynamically.

---

## 8. Payment Gateway & Atomic Enrollment

### Payment Gateway Architecture

- Accessible at `/checkout/:courseId`.
- Completely removed all debug/appsettings notices to present a clean, high-trust production interface.
- Supported Payment Modes:

1. **Credit / Debit Cards**: Includes a one-click **Test Card Fill** button ( `4242 4242 ...` ) for demonstrations.
2. **UPI / QR Code**: Mock QR code display with support for Google Pay, PhonePe, Paytm, or manual VPA entry.
3. **Net Banking**: Direct bank select chips for HDFC, ICICI, SBI, and Axis Bank.

- **Atomic Access Provisioning**: Once the transaction completes, the frontend sends an enrollment request to `POST /api/Courses/{courseId}/enroll`. An `Enrollment` record is committed, unlocking immediate access to the course player and redirecting to the congratulations screen.
- **Key Files**:
  - payment-gateway.ts
  - payment-gateway.html
  - enrollment-success.html

---

## 9. Google Gemini AI Copilot Integration

SkillHub integrates Google Gemini AI directly into the student experience via a floating, interactive copilot component.

```
[Student Browsing UI / Player]
            |
            ▼ (Sends prompt + current URL route)
[AiCopilotComponent (Angular)]
            |
            ▼ (HTTP POST /api/AiChat/message)
[AiChatController (ASP.NET Core)]
            |
            ▼
[GeminiAiService (Infrastructure)]
            |
            ▼ (REST call with API Key)
[Google Gemini Generative AI API (gemini-flash-latest)]
```

### Key Technical Capabilities:

1. **Context-Aware Recommendations**:
   - The frontend passes `contextUrl: this.router.url`.
   - `GeminiAiService.cs` inspects the route (e.g. `/courses/5` vs `/learn/5` vs `/admin/support`) and tunes its prompt and quick suggestion chips to match what the student is looking at.
2. **Dual-Tier Quota Management**:
   - **Guests**: Rate-limited to 5 questions per session using an in-memory thread-safe `ConcurrentDictionary<string, int>` tracker keyed by IP address and guest session UUID.
   - **Authenticated Students**: Enjoy unlimited AI learning queries.
3. **Key Files**:

- Backend: GeminiAiService.cs, AiChatController.cs
- Frontend: ai-copilot.ts, ai-copilot.service.ts

---

## 10. Advanced Technologies & Frameworks Explained

### 1. AutoMapper

- **Why We Use It**: Decouples database entities from public API responses.
- **Problem It Solves**: Without AutoMapper, developers must write dozens of manual mappings ( `dto.Title = entity.Title; dto.InstructorName = entity.Instructor.FirstName + ...` ). This is error-prone, repetitive, and risks exposing sensitive fields (such as `PasswordHash` ).
- **Location**: MappingProfile.cs.

### 2. FluentValidation

- **Why We Use It**: Provides a clean, fluent syntax for building strongly typed validation rules without cluttering entity classes or DTOs with data annotations.
- **Rule Example**: `RuleFor(x => x.Email).NotEmpty().EmailAddress();`
- **Location**: `SkillHub.Application/Features/*/Validators/`.

### 3. BCrypt Password Hashing ( `BCrypt.Net-Next` )

- **Why We Use It**: Employs adaptive salt-based hashing with configurable work factor (cost). Immune to rainbow table attacks and brute force cracking.
- **Location**: UserSeeder.cs, AuthService.cs.

### 4. Angular 20 Standalone Components & Signals

- Eliminates legacy `NgModule` boilerplate.

- Promotes modularity, faster tree-shaking, smaller bundle sizes, and clean lazy loading.
- Functional HTTP interceptor ( `jwtInterceptor` ) automatically injects Bearer authorization headers and intercepts 401s.

## 11. Complete Codebase File Index

| Module / Topic | Backend File (ASP.NET Core) | Frontend File (Angular) |
|---|---|---|
| **Authentication & JWT** | `SkillHub.API/Controllers/AuthController.cs`<br>`SkillHub.Infrastructure/Services/Identity/AuthService.cs` | `src/app/features/auth/login/login.ts`<br>`src/app/core/interceptors/jwt.interceptor.ts` |
| **Clean Architecture Root** | `SkillHub.Domain/Entities/`<br>`SkillHub.Application/Mappings/MappingProfile.cs` | `src/app/app.routes.ts` |
| **Database & Context** | `SkillHub.Infrastructure/Persistence/Context/ApplicationDbContext.cs` | — |
| **Seed Data (Users/Roles)** | `SkillHub.Infrastructure/seed/UserSeeder.cs`<br>`SkillHub.Infrastructure/seed/RoleSeeder.cs` | — |
| **Permission Studio** | `SkillHub.API/Controllers/PermissionController.cs`<br>`SkillHub.Infrastructure/Services/Admin/PermissionService.cs` | `src/app/features/admin/admin-permissions/admin-permissions.ts`<br>`src/app/features/admin/services/permission.service.ts` |
| **Course Authoring** | `SkillHub.API/Controllers/CourseController.cs`<br>`SkillHub.Infrastructure/Services/Courses/CourseService.cs` | `src/app/features/courses/course-create/course-create.ts`<br>`src/app/features/courses/course-list/course-list.ts` |
| **Course Player & PDFs** | `SkillHub.API/Controllers/LearningController.cs`<br>`SkillHub.Infrastructure/Services/Learning/LearningService.cs` | `src/app/features/enrollment/course-player/course-player.ts`<br>`src/app/features/enrollment/course-player/course-player.html` |
| **Quizzes & 70% Certificate** | `SkillHub.API/Controllers/QuizzesController.cs`<br>`SkillHub.Infrastructure/Services/Assessments/QuizService.cs`<br>`SkillHub.Domain/Entities/Learning/Certificate.cs` | `src/app/features/enrollment/course-player/course-player.ts`<br>`src/app/features/enrollment/models/quiz.model.ts` |
| **Helpdesk & Support** | `SkillHub.API/Controllers/SupportController.cs`<br>`SkillHub.Infrastructure/Services/Support/SupportService.cs` | `src/app/features/admin/admin-support/admin-support.ts`<br>`src/app/features/support/services/support.service.ts` |
| **Payment Gateway** | `SkillHub.API/Controllers/PaymentsController.cs`<br>`SkillHub.Infrastructure/Services/Payments/PaymentService.cs` | `src/app/features/enrollment/payment-gateway/payment-gateway.ts`<br>`src/app/features/enrollment/payment-gateway/payment-gateway.html` |
| **Google Gemini AI** | `SkillHub.API/Controllers/AiChatController.cs`<br>`SkillHub.Infrastructure/Services/Ai/GeminiAiService.cs` | `src/app/features/ai-copilot/ai-copilot.ts`<br>`src/app/features/ai-copilot/services/ai-copilot.service.ts` |

## 12. Viva / Practical Exam Top Questions & Answers

### Q1: What is Clean Architecture and why did you choose it over N-Tier architecture?

> **Answer**: Clean Architecture organizes software into concentric layers where dependencies strictly point inward towards the core Domain ( `SkillHub.Domain` ). In traditional N-Tier architectures, business logic often becomes tightly coupled to Entity Framework or database tables. In Clean Architecture, our business rules ( `Application` layer) and domain models ( `Domain` layer) are completely independent of UI technologies, third-party libraries, and database providers.

### Q2: How does Angular communicate securely with ASP.NET Core?

> **Answer**: Angular communicates with the Web API using JSON over HTTPS. When a user logs in, the backend issues a signed **JWT (JSON Web Token)** containing identity claims ( `userId` , `email` , `role` ). Angular stores this token securely and utilizes a functional **HTTP Interceptor** ( `jwt.interceptor.ts` ) that automatically attaches the `Authorization: Bearer <token>` header to all outgoing requests. The backend validates the signature, issuer, and expiration on every request.

### Q3: Explain how your 70% certificate rule is enforced in code.

> **Answer**: In `SkillHub.Infrastructure.Services.Learning.LearningService.cs`, when a lesson is completed, the system calls `EvaluateCertificateEligibilityAsync`. It executes two checks:
>
> 1. Counts completed lessons against total section lessons ( `completedLessons ≥ totalLessons` ).
> 2. Queries `QuizAttempts` for the student on that course and verifies that for every quiz, `bestAttempt.ScorePercentage ≥ 70`. Only when both conditions evaluate to `true` does the backend generate a unique `Certificate` record with a verifiable UUID code ( `SKILLHUB-CERT-XXXXXXXX` ).

### Q4: Why is SuperAdmin restricted to read-only oversight on support tickets?

> **Answer**: In real-world enterprise architectures, ticket handling is an operational responsibility delegated to **Product Support** and assigned admins ( `SubAdmin` ). The **SuperAdmin** role represents platform ownership and auditing. Giving SuperAdmin read-only oversight ensures separation of duties: SuperAdmin can monitor platform SLAs and customer issues without accidentally tampering with active ticket triage or assignments. This is enforced with a `403 Forbidden` guard in `SupportController.cs`.

### Q5: What is the purpose of AutoMapper and FluentValidation in your project?

> **Answer**:
>
> - **AutoMapper** prevents boilerplate code and eliminates "over-posting" vulnerabilities by transforming database entities into lightweight DTOs before sending them to the client.
> - **FluentValidation** decouples validation logic from model definitions. Instead of using messy attributes, we write expressive, testable validation rules in dedicated validator classes that execute before controller actions run.

### Q6: How does the Google Gemini AI integration handle guest usage versus student usage?

> **Answer**: `GeminiAiService.cs` tracks unauthenticated guest sessions in a thread-safe `ConcurrentDictionary` by IP and guest session ID, capping them at 5 exploratory queries to prevent API abuse. Authenticated students are granted unlimited queries and receive contextual suggestions tuned to their active route.

---

## 13. System Demo User Accounts

| Role | Email | Password | Capabilities to Demonstrate |
|---|---|---|---|
| **SuperAdmin** | `superadmin@skillhub.com` | `SuperAdmin123!` | User management, Permission Studio, Course Approvals, Read-Only Support Oversight |
| **ProductSupport** | `support@skillhub.com` | `Support123!` | Helpdesk Command: Triage tickets, assign to SubAdmins, resolve tickets |
| **SubAdmin** | `subadmin@skillhub.com` | `SubAdmin123!` | Operational Admin: Resolve assigned tickets, course management |
| **Instructor** | `instructor@skillhub.com` | `Instructor123!` | Authoring courses, managing sections, uploading lesson videos and PDFs |
| **Student** | `student@skillhub.com` | `Student123!` | Enrolling, course player, taking quizzes, earning certificates, raising tickets |

---

*Created for SkillHub Project Presentation, Practical Examination & Viva.*